

Analisi di Co-occorrenza di Terremoti con Scala e Apache Spark

Progetto — Scalable and Cloud Programming, A.A. 2025-26

[Nome Cognome] — [Matricola]

Repository: <https://github.com/Sh0wBlack32/SCP-earthquake-cooccurrence>

1. Descrizione del problema

L'obiettivo del progetto è realizzare, in Scala e Apache Spark, un'analisi di co-occorrenza di eventi sismici in località diverse. Data una serie storica di terremoti, ciascuno caratterizzato da latitudine, longitudine e istante temporale, si vuole individuare la coppia di località che co-occorrono più spesso, ovvero la coppia di aree in cui si registrano eventi sismici nella stessa finestra temporale (un giorno) nel maggior numero di giorni distinti. Il risultato richiesto è la coppia vincente e l'elenco, in ordine crescente, delle date in cui i due eventi co-occorrono.

Poiché la posizione esatta di un evento non è rilevante quanto l'area in cui esso si verifica, latitudine e longitudine vengono arrotondate alla prima cifra decimale (arrotondamento al valore più vicino). Questo introduce duplicati — più eventi nello stesso giorno che ricadono nella stessa "cella" geografica arrotondata — che devono essere deduplicati prima di generare le coppie, altrimenti si rischia di individuare come vincente una coppia di eventi nella stessa cella geografica.

1.1 Dataset

Il dataset è composto da record (latitude, longitude, date) in formato CSV con intestazione. Sono disponibili due versioni: una versione ridotta, usata per i test in locale e sulla configurazione solo-master, e una versione completa, usata per l'esecuzione e la valutazione delle prestazioni su Google Cloud DataProc.

2. Approccio implementativo

L'elaborazione segue lo schema map-reduce ed è interamente distribuita tramite RDD di Spark. La pipeline comune ai due approcci è la seguente:

- lettura del CSV e arrotondamento di lat/lon alla prima cifra decimale;
- troncamento del timestamp alla data (YYYY-MM-DD);
- deduplica delle coppie (località, data) generate dall'arrotondamento;
- generazione, per ogni giorno, di tutte le coppie canoniche di località (loc_i, loc_j) con $loc_i < loc_j$;
- deduplica delle coppie (coppia, data);
- conteggio delle co-occorrenze per coppia e individuazione del massimo;
- raccolta e ordinamento delle date per la coppia vincente.

2.1 Approccio 1 — groupByKey

Le località vengono raggruppate per data con **groupByKey**; per ogni giorno, le coppie candidate vengono generate localmente (in memoria, sul singolo executor) a partire dalla lista di località distinte osservate quel giorno. È l'approccio più efficiente quando il numero di eventi per giorno è contenuto, poiché evita uno shuffle aggiuntivo per generare le coppie.

2.2 Approccio 2 — Self-Join

In alternativa, le coppie vengono generate tramite un self-join distribuito dell'RDD chiave-valore (data, località) con se stesso, filtrando poi le coppie con $loc1 < loc2$ per eliminare simmetrie e auto-coppie. Questo approccio distribuisce il lavoro di generazione delle coppie su tutto il cluster invece di concentrarlo sul singolo executor che gestisce un dato giorno, a costo di uno shuffle più oneroso; può risultare preferibile quando i giorni con

molti eventi (skew) rendono costoso il groupByKey.

[NOTA: qui puoi aggiungere 3-4 righe sulle differenze osservate concretamente tra i due approcci durante l'esecuzione — ad esempio, se uno dei due ha sofferto più shuffle spill, se uno dei due non è riuscito a completare su cluster piccoli, ecc. — sulla base di quanto osservato nella Sezione 4.]

3. Setup sperimentale

Le prove sono state eseguite su Google Cloud DataProc, con macchine di tipo **n2-standard-4** sia per il nodo master sia per i nodi worker (configurazione obbligatoria per poter creare cluster con 4 worker), boot disk da 240 GB per contenere l'uso entro i limiti degli education credit, e cluster di 2, 3 e 4 worker. Il dataset completo e il JAR dell'applicazione sono stati caricati su un bucket Google Cloud Storage e referenziati tramite URI gs:// nei job Spark.

[NOTA: se hai anche testato diverse configurazioni di partizionamento (es. repartition(16), repartition(32) sul cluster a 4 worker), descrivile qui con una frase — quali valori, e perché quei valori.]

Versioni e librerie

Componente	Versione
Scala	2.12.17
Apache Spark	3.3.2
sbt	1.9.7
Piattaforma	Google Cloud DataProc

4. Analisi di scalabilità e prestazioni

[COMPLETARE CON I DATI REALI DOPO LE PROVE SU DATAPROC. La tabella sottostante è un modello: riportare, per ogni combinazione worker/approccio/partizioni, il tempo di esecuzione misurato (ad es. dai log del job o dalla Spark UI/history server).]

Worker	Partizioni	Approccio	Tempo (s)
2	default	groupByKey	
2	default	self-join	
3	default	groupByKey	
3	default	self-join	
4	default	groupByKey	
4	default	self-join	
4	16	groupByKey	
4	32	groupByKey	

4.1 Scalabilità (strong scaling)

[COMPLETARE: commentare l'andamento del tempo di esecuzione al crescere del numero di worker a parità di dataset (strong scaling). Calcolare lo speedup $S(n) = T(1 \text{ unità di riferimento})/T(n)$ e, se rilevante, l'efficienza $E(n) = S(n)/n$. Discutere se la scalabilità è vicina a quella ideale o se si osserva un plateau, e ipotizzare le cause (overhead di shuffle, skew dei dati, costo di comunicazione di rete tra i nodi, ecc.).]

4.2 Effetto del partizionamento

[COMPLETARE: confrontare i tempi con partizionamento di default rispetto a repartition(16)/repartition(32) sul cluster a 4 worker. Discutere se un numero maggiore di partizioni ha aiutato il bilanciamento del carico o se ha introdotto overhead di scheduling eccessivo.]

4.3 Confronto tra i due approcci

[COMPLETARE: quale approccio (groupByKey o self-join) è risultato più efficiente nelle prove, e perché, sulla base di quanto discusso in Sezione 2.]

5. Conclusioni

[COMPLETARE: 3-5 righe di sintesi — cosa ha funzionato, quale approccio si è dimostrato preferibile nel caso in esame, quali limiti o miglioramenti futuri (es. uso di DataFrame/Dataset invece di RDD, broadcast join per dataset più piccoli, tuning aggiuntivo dei parametri Spark).]

Codice sorgente: <https://github.com/Sh0wBlack32/SCP-earthquake-cooccurrence> (istruzioni per l'esecuzione su DataProc nel README del repository).